Creating Classes

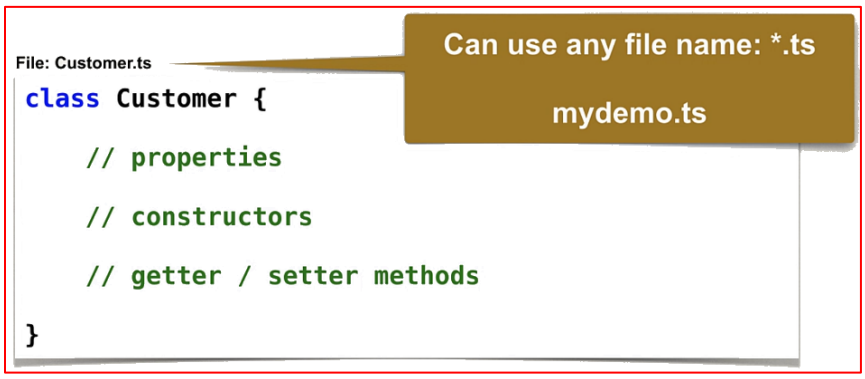
In this lesson, we will explore the essentials of creating classes in TypeScript, including the utilization of constructors, getters, and setters. Let's begin by exploring a basic example that involves a customer class.

Example: Defining a Customer Class

A `customer` can have a `first name` and `last name`, both of type string. Here's a structural outline of how a class is designed in TypeScript:

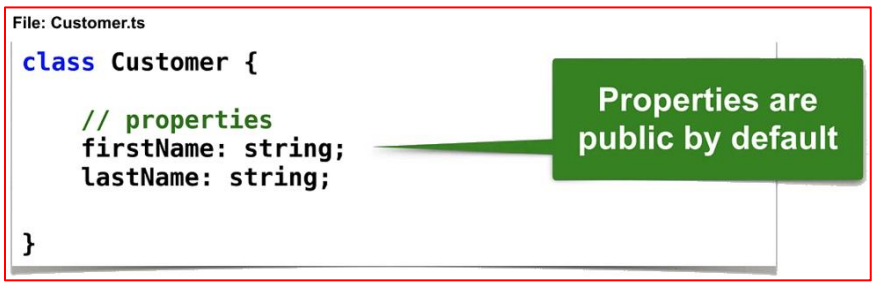
- **Keyword:** Use the "class" keyword followed by the name of the class.
- **Properties:** Define properties like first and last name.
- **Constructors:** Learn how to create constructors to initialize properties.
- **Getters and Setters:** Implement methods for accessing and modifying properties.

This code can be placed in a file, such as `"Customer.ts"`. Though you can use any file name as long as it ends with a dot extension, by convention, I prefer to use the actual name of the class. It's not a requirement, but rather a common practice.



Working with the Customer Class

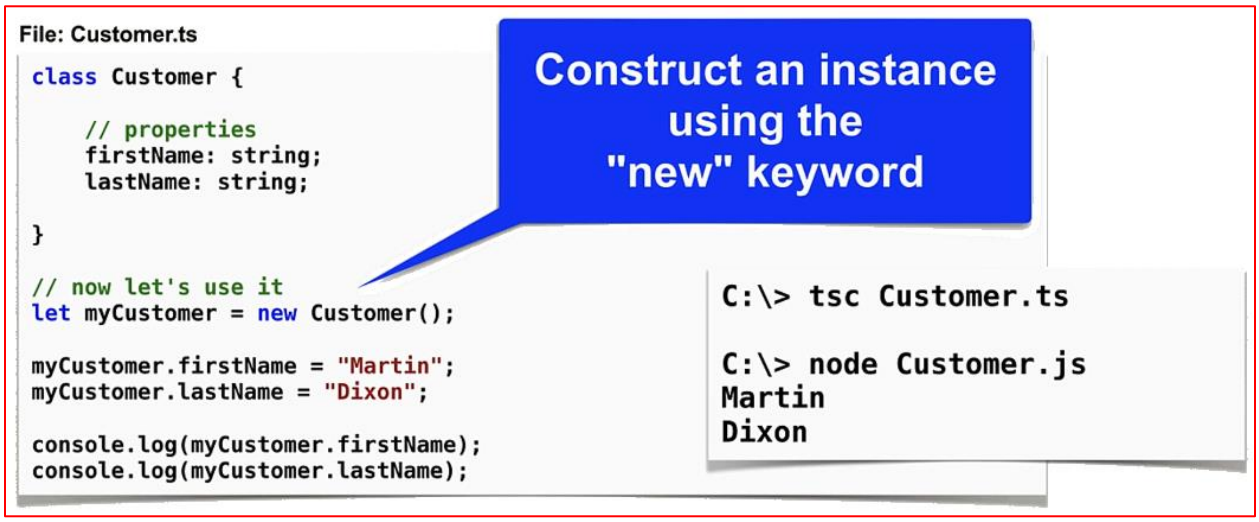
Let's now define some properties for our customer class, having two public properties (public by default) - first name and last name of type string. We will cover access modifiers like public, protected, and private later in this lesson, but for now, let's stick to the basics.



Once the customer class is defined, we can create an instance using the `new` keyword:

```
let myCustomer = new Customer();
myCustomer.firstName = "Martin";
myCustomer.lastName = "Dixon";
```

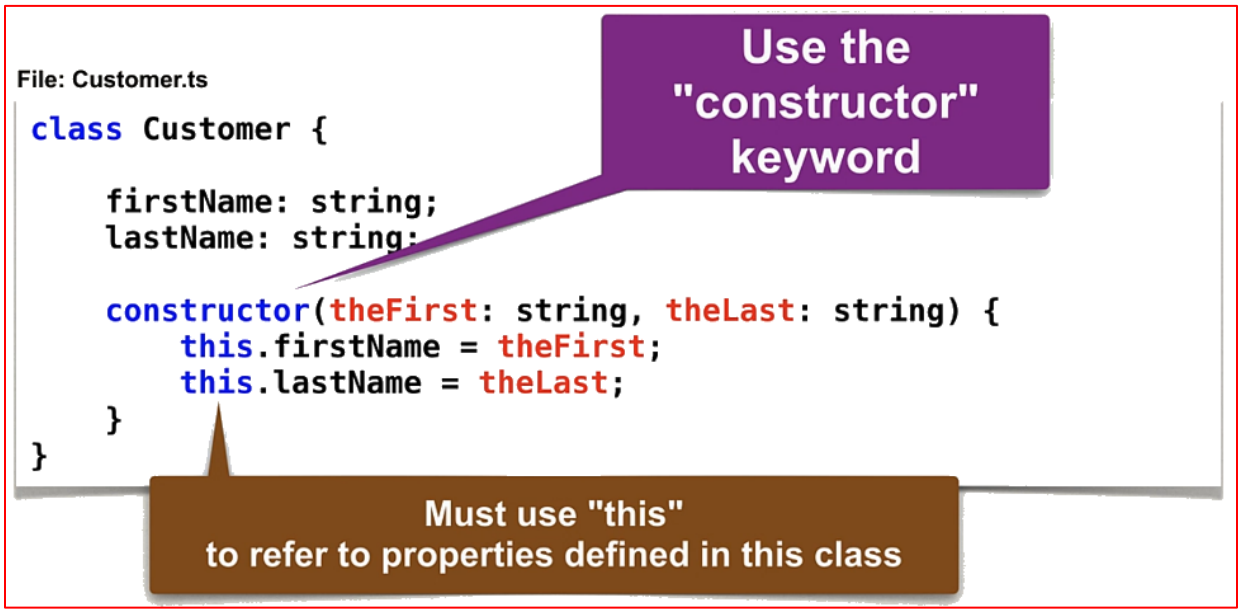
Then, by compiling and running the code, we'll see the output of "Martin Dixon".



Creating a Constructor

To create a constructor, utilize the `constructor` keyword, passing parameters for the first and last name, and assign them accordingly:

```
this.firstName = firstName;
this.lastName = lastName;
```



We must use `this` to refer to properties defined in the class. By updating our example to use this new constructor, we will observe the same output, but with a more streamlined initialization process.

File: Customer.ts

```
class Customer {

    firstName: string;
    lastName: string;

    constructor(theFirst: string, theLast: string) {
        this.firstName = theFirst;
        this.lastName = theLast;
    }
}

// now let's use it
let myCustomer = new Customer("Martin", "Dixon");

console.log(myCustomer.firstName);
console.log(myCustomer.lastName);
```

Construct an instance using our new constructor

```
C:\> tsc Customer.ts

C:\> node Customer.js
Martin
Dixon
```

Access Modifiers

Access modifiers define the accessibility of a property:

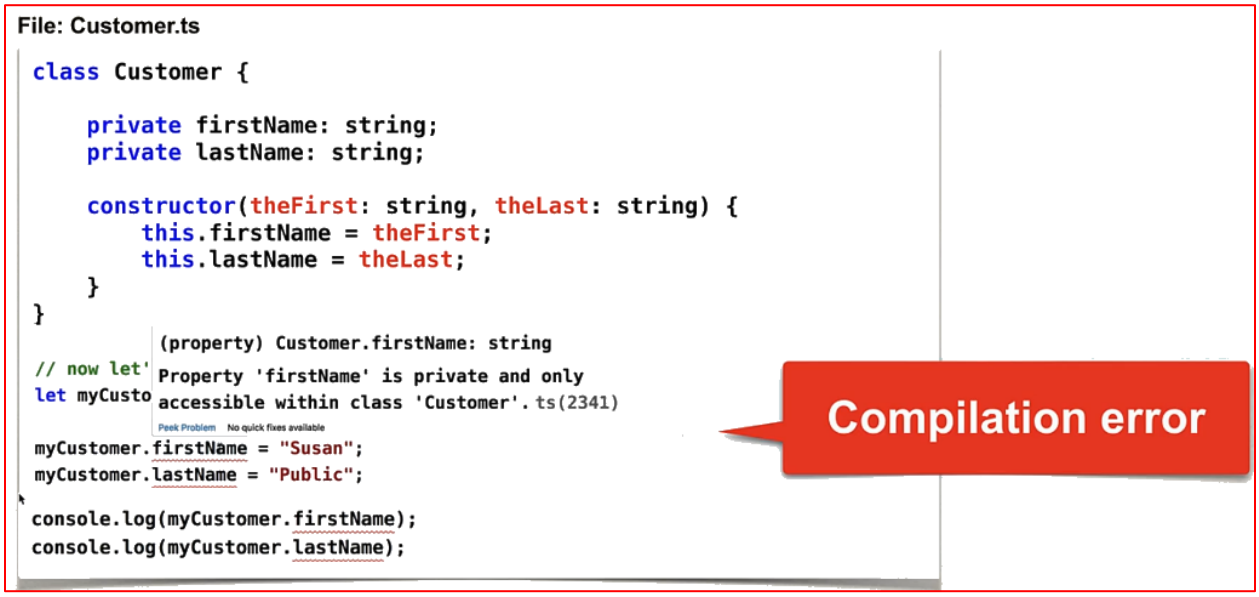
- **Public:** Accessible to all classes and is the default modifier.
- **Protected:** Only accessible in the current class and subclasses.
- **Private:** Only accessible in the current class.

If we mark the properties as private, we'll encounter compilation errors when trying to access them. The Integrated Development Environment (IDE) will indicate that properties like first name are private and only accessible within the class "customer".

Modifier	Definition
`public`	Property is accessible to all classes (default modifier)
`protected`	Property is only accessible in current class and subclasses
`private`	Property is only accessible in current class

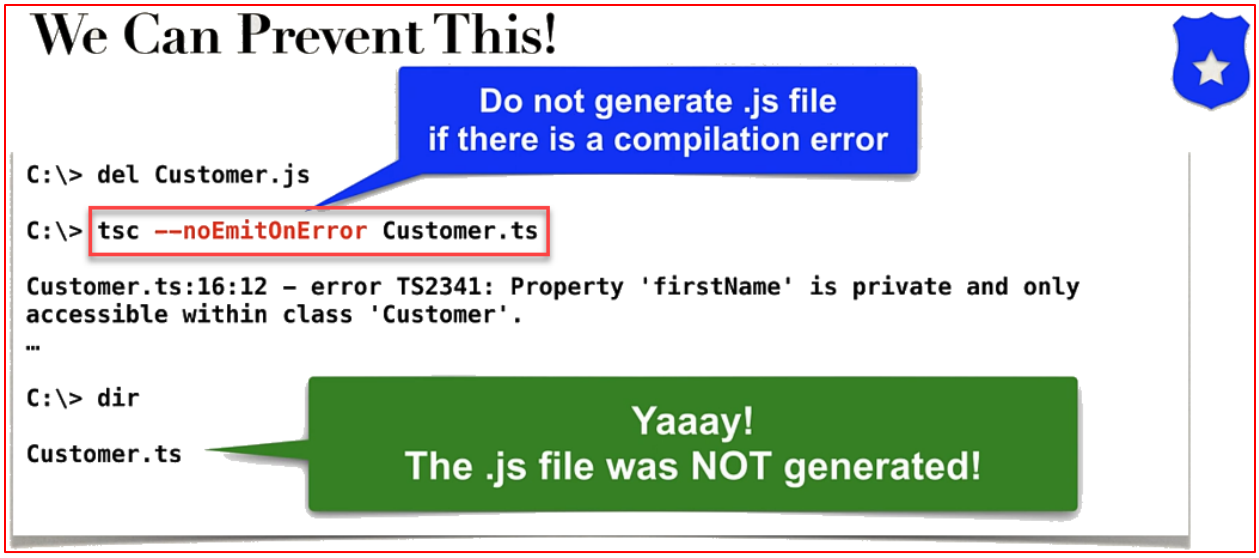
Handling Compilation Errors

Here's an intriguing aspect of TypeScript: even with compilation errors, it still generates a `.js` file. If you compile and then inspect the directory contents, you'll find a "customer.js" file. This might seem illogical since there was a compiler error. It can even run the code, leading to unexpected outputs.





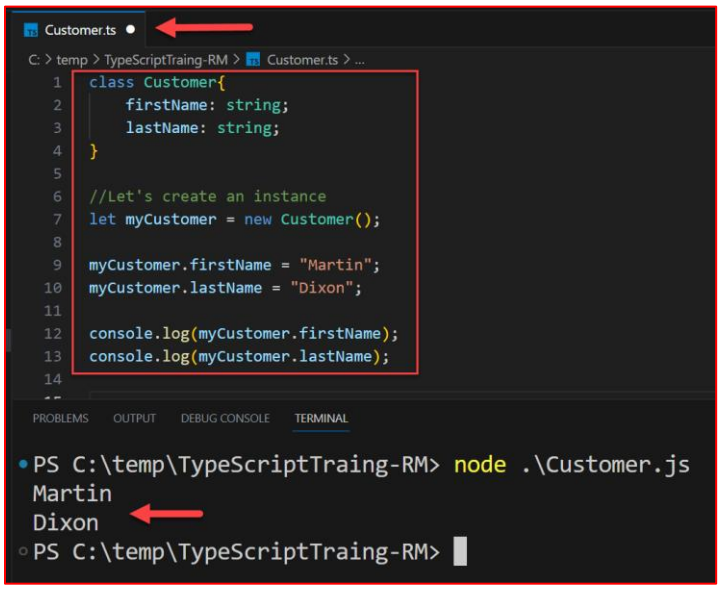
We can prevent this by using a compiler flag `--noEmitOnError`, which says not to generate a `.js` file if there's a compilation error. This way, we can manage the TypeScript compiler effectively.

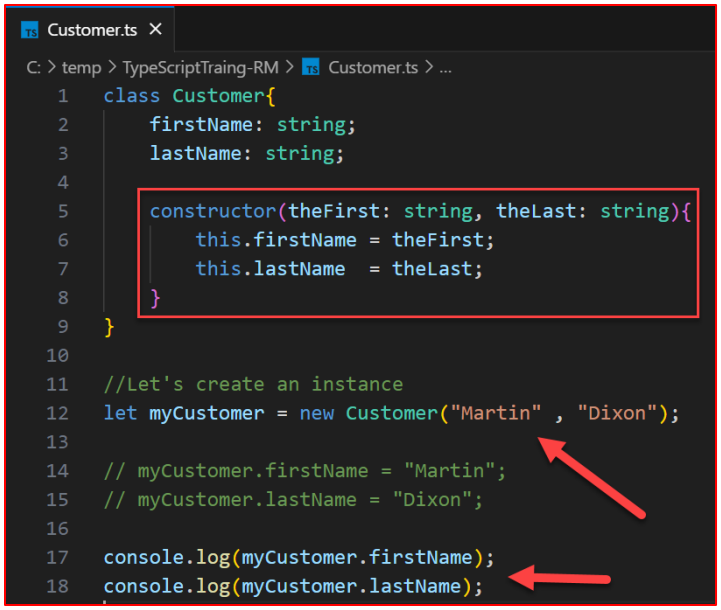


Building the Code in an IDE

Now, let's move into our Integrated Development Environment (IDE) and write the code for our customer class to test it out.

1. **Set Up Directories:** Create main and subfolders to organize our code.
2. **Create a New File:** Name it "Customer.ts" to house our customer class.
3. **Define the Customer Class:** Outline properties like first name and last name, remembering that all properties are public by default.
4. **Create an Instance:** Utilize the `new` keyword, set properties, and display them using `console.log`.
5. **Compile and Run:** Use the appropriate commands to see the output of "Martin Dixon".
6. **Use a Constructor:** Adjust the code to define and use a constructor, passing first and last name directly.





By following these steps, we can create, utilize, and adapt our customer class with ease.